

1 目录结构说明

- ├─cfg 配置文件目录，该目录保存了分类网络和预测网络
- ├─exp_result 实验结果输出的目录
- ├─log
- ├─├─classify_net 分类网络训练记录和模型存储目录
- ├─├─predict_net 预测网络训练记录和模型存储目录
- ├─Stk_Day_QFQ 实验数据目录
- ├─train_data 训练数据目录
- ├─├─Pattern 自定义Pattern保存目录

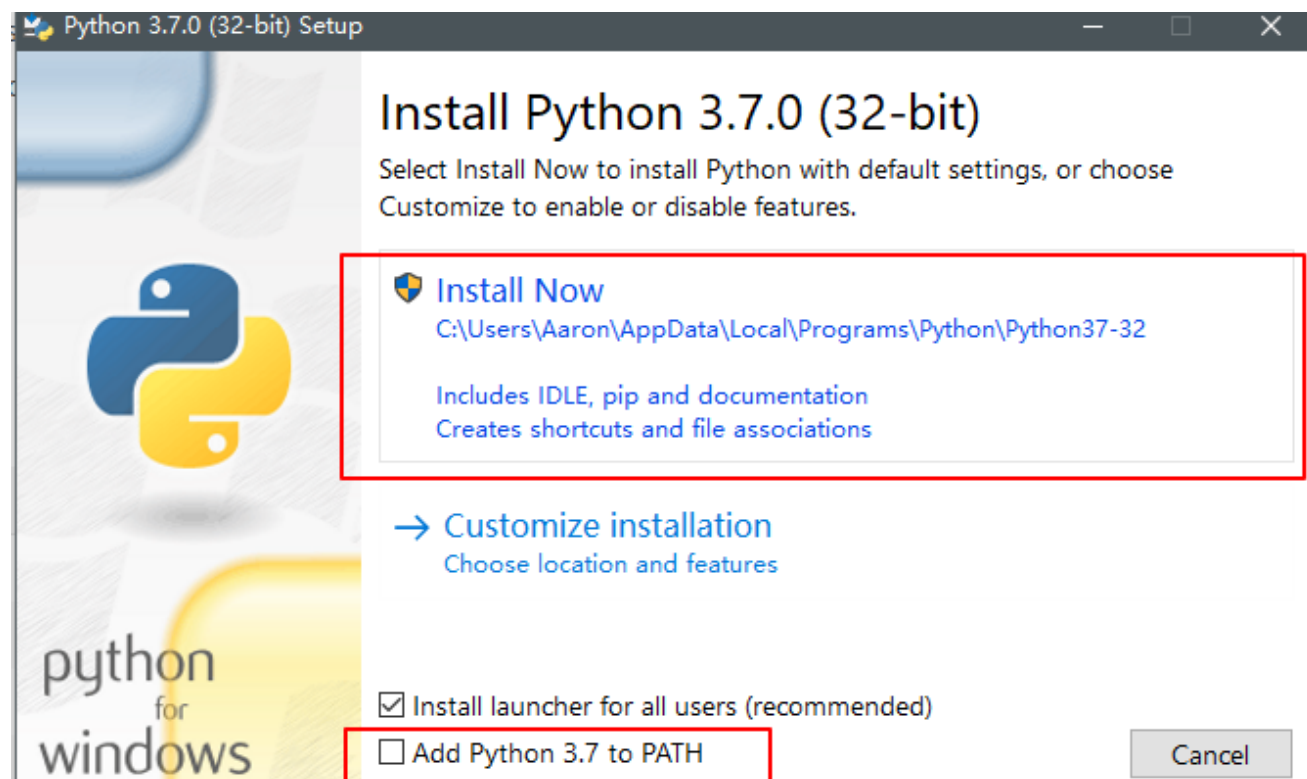
运行环境配置说明

这里的配置方法只针对windows，mac和Linux下的配置方法大同小异，由于我这边没有相应的环境，暂不演示。

1.1 Python 3.6+

[下载链接](#)

安装的时候请勾选 Add Python to PATH



1.2 安装所需的第三方包

项目目录下，打开console，输入

```
pip install -r requirements.txt
```

这一步依网络状况不同需要等待的时间也不同，请耐心等待。

```
(test_env) C:\Users\Aaron\Desktop>cd stock
(test_env) C:\Users\Aaron\Desktop\stock>ls
PatternGenerator.py  Roadmap.pdf  evaluate.py  model.py  stock.zip
README.md           Stk_Day_QFQ  exp.ipynb   net_config_loader.py  train.ipynb
README.pdf          __pycache__  exp_result  report    train_classify_net.py
Roadmap.md          cfg          log         requirements.txt  train_data

(test_env) C:\Users\Aaron\Desktop\stock>pip install -r requirements.txt -i https://pypi.douban.com/simple
Looking in indexes: https://pypi.douban.com/simple
Collecting PyQt5 (from -r requirements.txt (line 1))
  Downloading https://pypi.doubanio.com/packages/5d/85/d174a50e0d6b60aa9113f6a32afb31f25345bec8584992af486235373252/PyQt5-5.11.2-5.11.1-cp35-cp36-cp37-cp38-none-win_amd64.whl (93.3MB)
    100% |#####| 93.4MB 2.6MB/s
Collecting tqdm (from -r requirements.txt (line 2))
  Downloading https://pypi.doubanio.com/packages/93/24/6ab1df969db228aed36a648a8959d1027099ce45fad67532b9673d533318/tqdm-4.23.4-py2.py3-none-any.whl (42kB)
    100% |#####| 51kB 1.5MB/s
Collecting numpy (from -r requirements.txt (line 3))
  Downloading https://pypi.doubanio.com/packages/0d/b7/0c804e0bcba6505f8392d042d5e333a5e06f308e019517111fbc7767a0bc/numpy-1.14.5-cp36-cp36m-win_amd64.whl (13.4MB)
    100% |#####| 13.4MB 815kB/s
Collecting pandas (from -r requirements.txt (line 4))
  Downloading https://pypi.doubanio.com/packages/07/54/5379878cd2ccabd08ab9ce356e204a5bb46c870f203c93c808c22dd63125/pandas-0.23.3-cp36-cp36m-win_amd64.whl (7.7MB)
    100% |#####| 7.7MB 905kB/s
Collecting matplotlib (from -r requirements.txt (line 5))
  Downloading https://pypi.doubanio.com/packages/bf/b9/485032835e979ee11d514bb3b9b0543a928b8b96c099c178aeab1d2ba861/matplotlib-2.2.2-cp36-cp36m-win_amd64.whl (8.7MB)
```

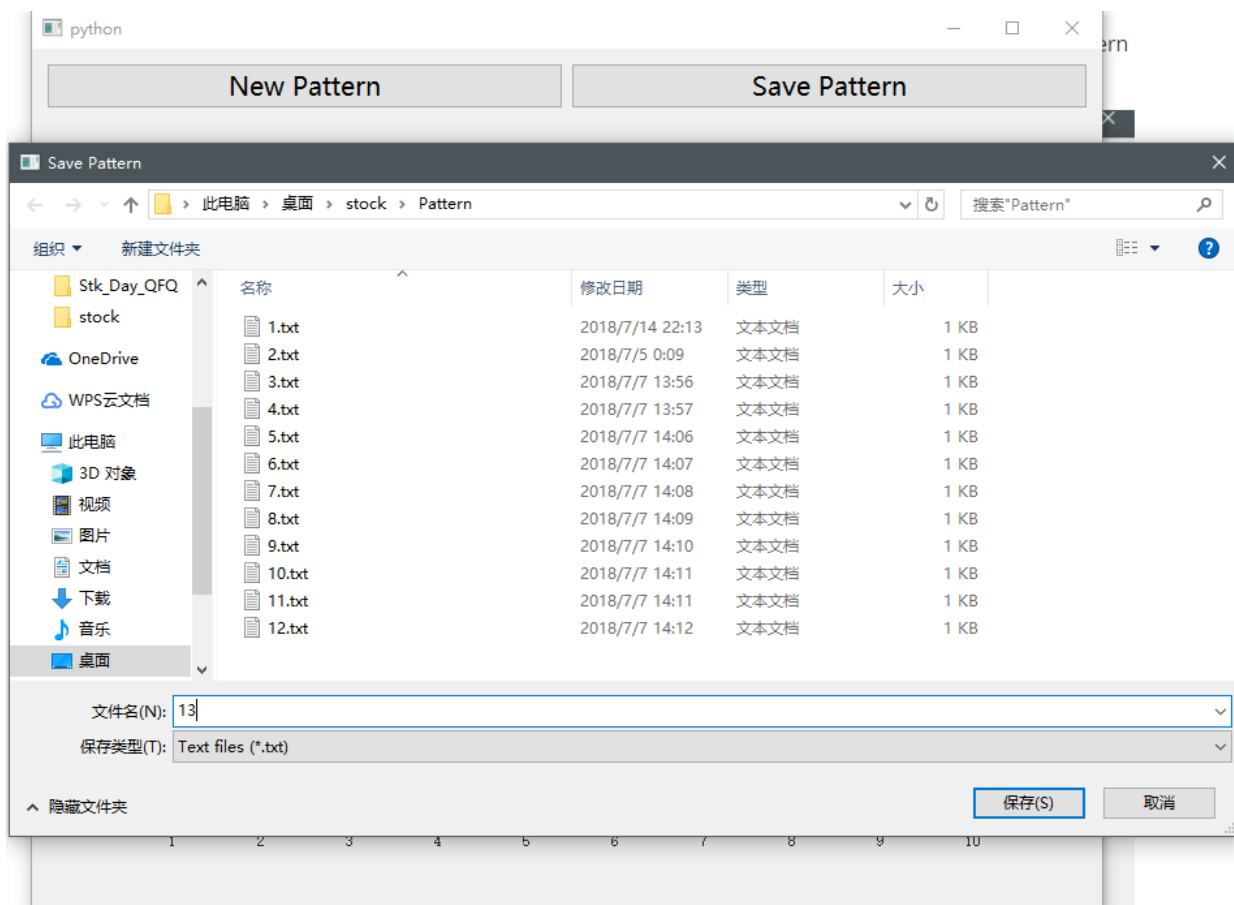
2 自定义Pattern

运行Pattern生成器

```
python PatternGenerator.py
```

Pattern生成器支持10个数据点，每个数据点可以鼠标左键点选拖动调整点位的值，调整完成后点击Save Pattern即可保存。





自定义的Pattern保存在Pattern目录下，编号依次增长，例如上图中，现在已有12个Pattern，新增一个Pattern可以命名为13

3 训练

3.1 网络结构调整

分类网络和预测网络的配置分别在cfg/classify_net.cfg, cfg/predict_net.cfg中，两个文件可以用任意文本编辑器打开，其中定义了输入层、隐藏层和输出层的神经元个数，以classify_net.cfg为例：

```
input 20
layer1 16
output 12
```

其输入层为20，隐藏层为16，输出层为12，**输出层的12神经元对应12个Pattern，因此如果有新增的自定义Pattern，例如现在有13个Pattern了，拿输出层的个数也要改为13，如果自定义Pattern数量较大，输出层神经元数量比较大，可以考虑适当增加隐藏层神经元的个数**

3.2 训练数据准备

训练数据都在Pattern这个文件夹中，Pattern文件夹中包含了以数字编号命名的Pattern数据和趋势数据。

- Pattern数据由Pattern生成器生成，准备数据时不用动；

- 趋势数据的文件名是trend.txt，其内容如下。其中0表示Up，1表示Down，每一行对应一个Pattern的后续趋势，这一部分与论文原文的Table 2。**需要注意的是，这里12行数据，严格对应Pattern1到Pattern12，Pattern文件保存的时候需要严格 [编号].txt的格式命名，否则会导致Pattern数据和其趋势定义不匹配的问题，导致后面的训练没有意义。如果有新增的自定义Pattern，trend.txt里也要新增一行**

```
0
1
0
0
1
1
0
1
0
1
0
0
```

3.3 训练

这里使用Python的notebook来一边训练一边展示训练的结果，这样比较方便查看；

首先在项目目录输入

```
jupyter-notebook
```

输入之后会自动打开浏览器加载一个网页如下，打开train.ipynb

Files	Running	Clusters
Select items to perform actions on them.		
Upload New ↻		
0 /		
Name Last Modified File size		
cfg	7 天前	
exp_result	14 小时前	
log	6 天前	
Pattern	11 分钟前	
report	15 小时前	
Stk_Day_QFQ	6 天前	
exp.ipynb	13 小时前	2.29 MB
train.ipynb	14 小时前	1.47 MB
evaluate.py	14 小时前	4.59 kB
model.py	15 小时前	7.18 kB
net_config_loader.py	7 天前	453 B
PatternGenerator.py	11 天前	6.23 kB
README.md	1 小时前	481 B
requirements.txt	18 小时前	48 B
Roadmap.md	11 天前	1.22 kB
Roadmap.pdf	12 天前	353 kB
stock.zip	18 小时前	498 kB
train_classify_net.py	17 小时前	997 B

选择kernel/Restart & Run All 即可开始一次训练

JupyterLab interface showing a notebook with code cells and a plot.

The top bar shows the JupyterLab logo, the current file name (Untitled1), the last checkpoint (上星期_02:42), and the autosave status (autosaved). The right side shows the Trusted status and the Python 3 kernel.

The notebook contains the following code cells:

```
In [1]: 1 from matplotlib import pyplot as plt
2 import numpy as np
3 import os
4 from model import ...
```

The context menu is open over the first cell, showing options: Interrupt, Restart, Restart & Clear Output, Restart & Run All (highlighted with a red box), Reconnect, Shutdown, and Change kernel.

```
In [2]: 1 #Load patterns
2 pattern_dir = 'Pattern'
3 pattern_list = []
4 pattern_files = sorted(os.listdir(pattern_dir), key=lambda x : int(x.split('.')[0]))
5 for file in pattern_files:
6     with open(os.path.join(pattern_dir, file)) as f:
7         pattern_list.append(np.array([float(x) for x in f.read().strip().split('\n')], dtype=np.float32))
8 patterns = np.vstack(pattern_list)
```

```
In [3]: 1 # Draw patterns
2 col = 4
3 row = int((patterns.shape[0] + col - 1) / col)
4 for i in range(patterns.shape[0]):
5     plt.subplot(row, col, i+1)
6     plt.plot(patterns[i])
7     plt.title(str(i+1))
8 plt.tight_layout()
```

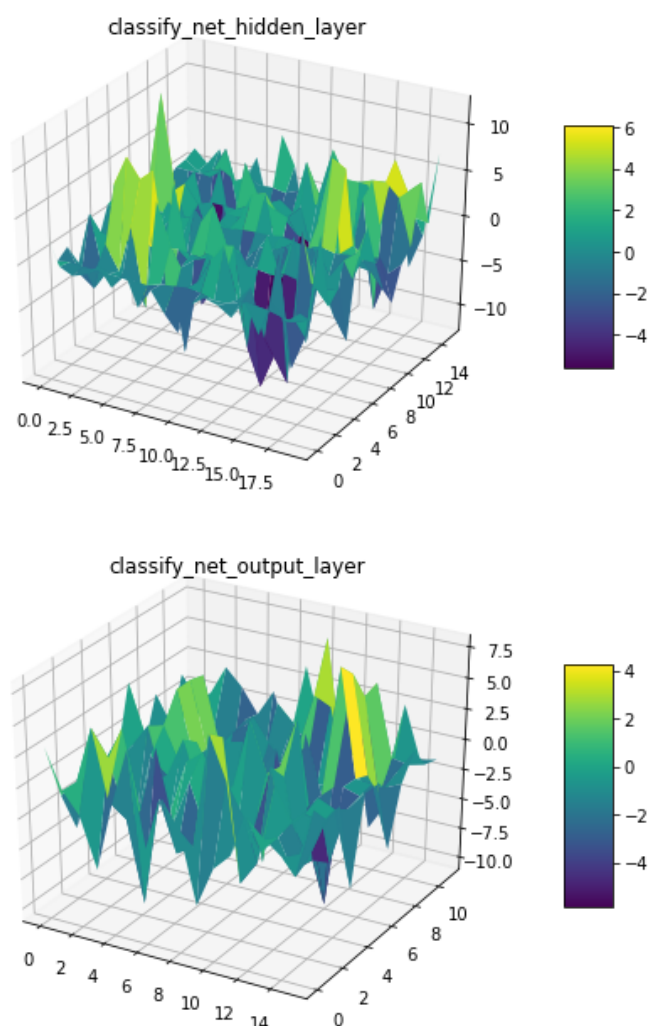
The output of the third cell is a 3x4 grid of 12 subplots, each showing a different pattern (labeled 1 to 12) plotted against a range of 0 to 5. Each plot has a y-axis ranging from 0 to 1.

```
In [4]: 1 X = np.concatenate((patterns, 1 - patterns), axis=1)
2 Y = np.array(range(X.shape[0])).reshape((12,))
3 classify_net = ClassifyNetowrk()
```

```
In [5]: 1 # Draw train set
```

运行完成后网页里会同时显示训练的结果

```
In [14]: 1 display_weights_in_3d(classify_net_weights[0].T, 'classify_net_hidden_layer')
2 display_weights_in_3d(classify_net_weights[1].T, 'classify_net_output_layer')
3 display_weights_in_3d(predict_net_weights[0].T, 'predict_net_hidden_layer')
4 display_weights_in_3d(predict_net_weights[1].T, 'output_net_hidden_layer')
```



训练参数调整

训练参数主要包括max_epoch和end_loss，即最大训练步数和训练终止条件

默认值定义在model.py文件的73行

```
def train(self, input, label, max_epoch=10000, verbose=False, end_loss=0.0):
```

如需更改，可以在notebook中添加代码指定，如

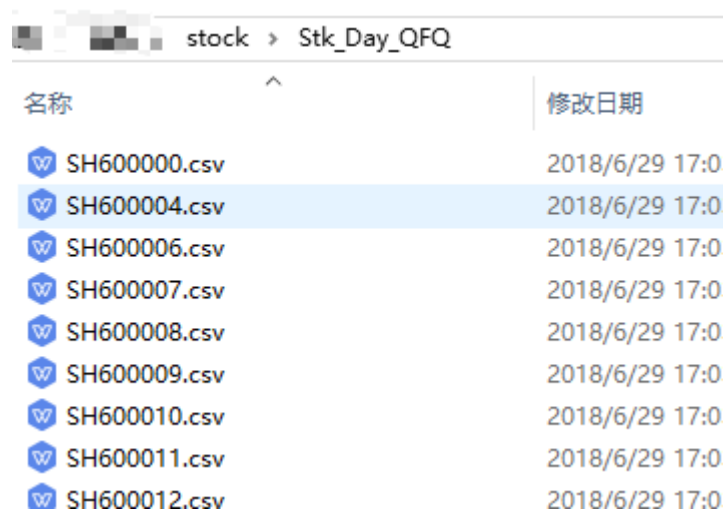
```
# train classify net
losses = classify_net.train(X, Y, verbose=True, max_epoch=5000, end_loss=0.07)
```

其他详细信息可以参考model.py文件中的代码注释

训练好的模型分别保存在log/classify_net和log/predict_net中，每次训练都会覆盖这两个文件夹，如需长时间保存某次训练的结果，可以复制这两个文件夹

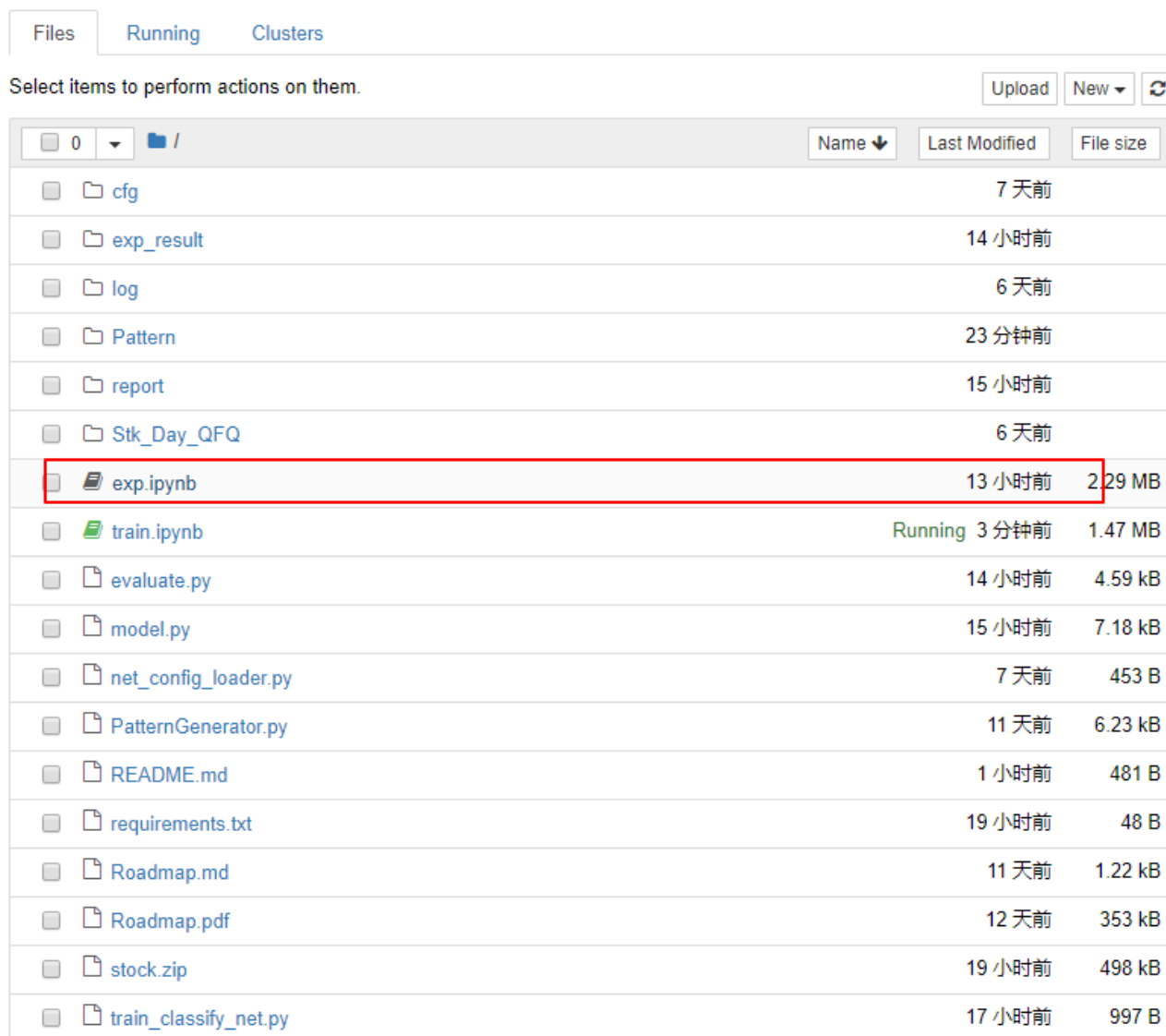
3.4 使用已训练好的模型对数据进行实验

预先将Stk_Day_QFQ 文件夹拷到项目目录下



名称	修改日期
SH600000.csv	2018/6/29 17:0
SH600004.csv	2018/6/29 17:0
SH600006.csv	2018/6/29 17:0
SH600007.csv	2018/6/29 17:0
SH600008.csv	2018/6/29 17:0
SH600009.csv	2018/6/29 17:0
SH600010.csv	2018/6/29 17:0
SH600011.csv	2018/6/29 17:0
SH600012.csv	2018/6/29 17:0

同样在notebook中，点击打开exp.ipynb

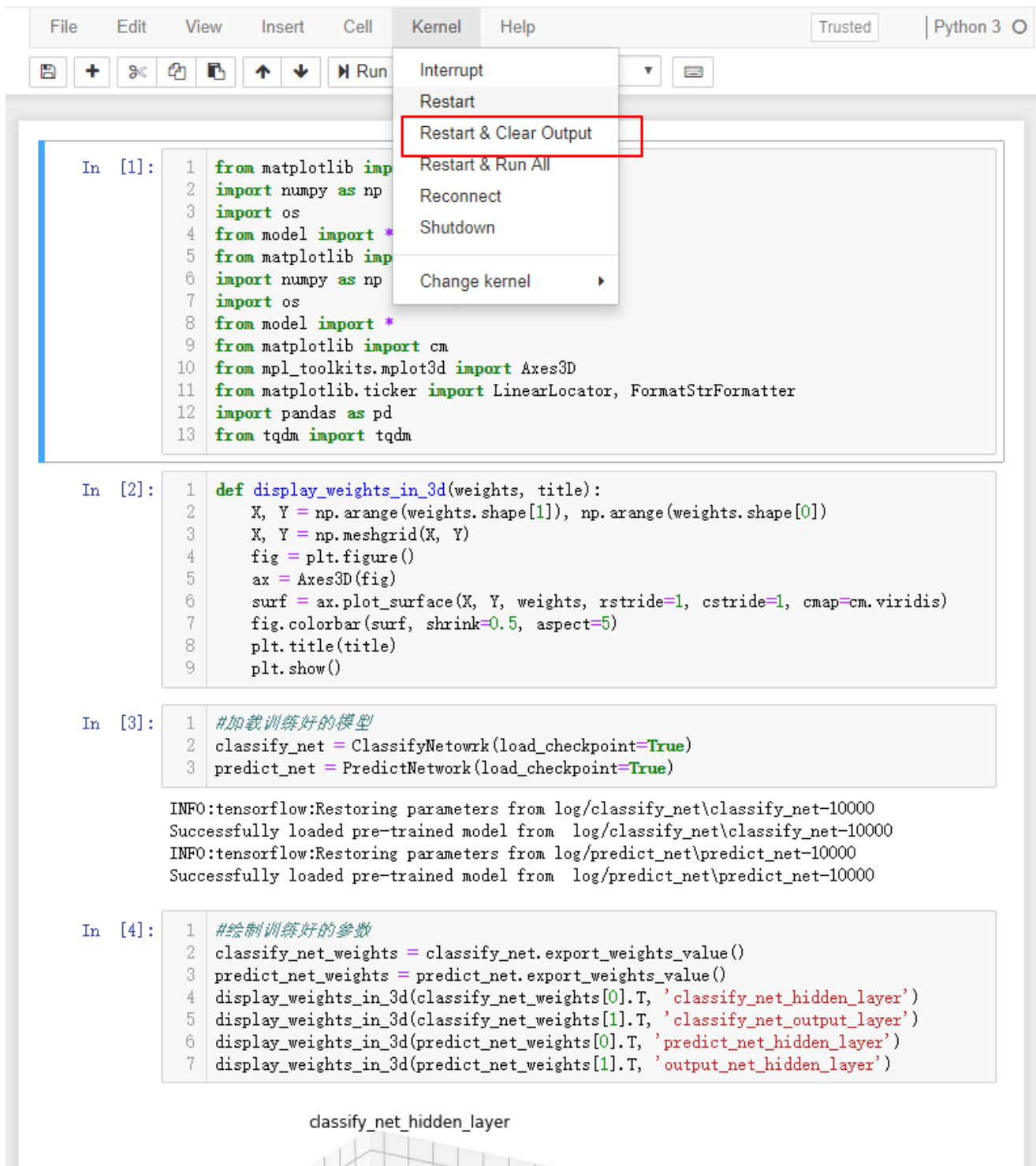


Files Running Clusters

Select items to perform actions on them. Upload New ↕

	Name ↓	Last Modified	File size
☐	cfg	7 天前	
☐	exp_result	14 小时前	
☐	log	6 天前	
☐	Pattern	23 分钟前	
☐	report	15 小时前	
☐	Stk_Day_QFQ	6 天前	
☐	exp.ipynb	13 小时前	2.29 MB
☐	train.ipynb	Running 3 分钟前	1.47 MB
☐	evaluate.py	14 小时前	4.59 kB
☐	model.py	15 小时前	7.18 kB
☐	net_config_loader.py	7 天前	453 B
☐	PatternGenerator.py	11 天前	6.23 kB
☐	README.md	1 小时前	481 B
☐	requirements.txt	19 小时前	48 B
☐	Roadmap.md	11 天前	1.22 kB
☐	Roadmap.pdf	12 天前	353 kB
☐	stock.zip	19 小时前	498 kB
☐	train_classify_net.py	17 小时前	997 B

选择kernel/Restart & Run All 即可用训练好的模型跑一遍所有的数据，

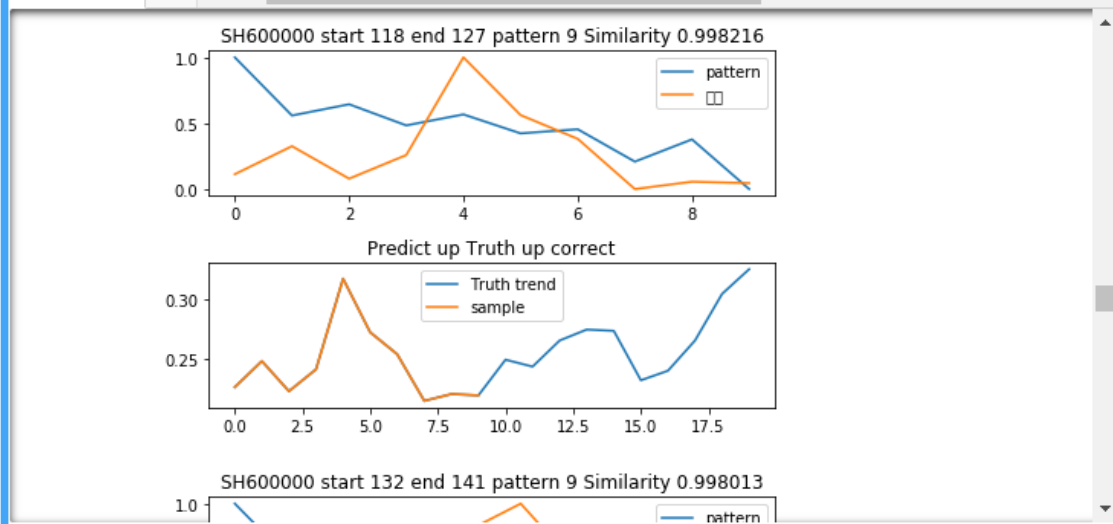


这个notebook执行到最后一步的时候，会绘制所有的相似度大于给定阈值的样本数据，时间会很长，如果不需要的话可以手动点击下图中的停止按钮停止，也可以选择注释掉代码。

```

34 计算样本实际的后续趋势
35 actual_trend = np.expand_dims(np.array([np.mean(data[int(i+1):int(i+11)]) > data[int(i)]
36
37 把这些结果存下来
38 df = pd.DataFrame(np.concatenate((start_idx, end_idx, recog_pattern, recog_pattern_similarity),
39                               columns=['start', 'end', 'Pattern类型', 'Pattern相似度', '预测趋势', '实际趋势'])
40
41 for i in range(actual_trend.shape[0]):
42     if recog_pattern_similarity[i] > similarity_threshold:
43         plt.figure(figsize=(8, 8))
44         plt.subplot(2,1,1)
45         plt.plot(patterns[recog_pattern[i, 0]])
46         plt.plot(norm_raw_values(single_stock_samples[i].reshape(1, 10))[0])
47         plt.legend(['pattern', 'sample'])
48         plt.title('{} start {} end {} pattern {} Similarity {:.2f}'.format(file.split('.')[0],
49                               recog_pattern_similarity[i, 0]))
50         plt.subplot(2,1,2)
51         plt.plot(data[int(start_idx[i]):int(start_idx[i])+20])
52         plt.plot(data[int(start_idx[i]):int(end_idx[i]+1)])
53         plt.title('Predict {} Truth {} {}'.format('up' if recog_trend[i, 0] > 0 else 'down',
54                               'up' if df['实际趋势'].values[i] > 0 else 'down',
55                               'correct' if recog_trend[i, 0] == df['实际趋势'].values[i] else 'wrong'))
56         plt.legend(['Truth trend', 'sample'])
57         plt.tight_layout()
58         plt.show()

```



In []: 1

加载特定的训练结果

```

1 #加载训练好的模型
2 classify_net = ClassifyNetowrk(load_checkpoint=True)
3 predict_net = PredictNetwork(load_checkpoint=True)

```

代码中定义神经网络变量的时候，设load_checkpoint=True，程序会自动从默认路径（log/classify_net和log/predict_net）加载已训练好的网络，如果要加载某个特定的训练结果，只需设置checkpoint_path的路径就行。

#加载训练好的模型

```

classify_net = ClassifyNetowrk(load_checkpoint=True, checkpoint_path='log/classify_net1')
predict_net = PredictNetwork(load_checkpoint=True, checkpoint_path='log/predict_net1')

```

写在后面

说明一下，代码里是base 0的，即从0开始数，因此1号pattern在代码里会是0号，客户在阅读代码的时候请注意。另外，所有的图表输出都是有按base1输出的。